SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

DAY 8

Software Engineering Principles & Software Characteristics

A student-facing chapter connecting foundational engineering principles to the quality and behavior of real software systems

Module I: Introduction to Software Engineering
Topics: Software Engineering Principles and Software Characteristics
Duration: 1 hour

Running example: College Event Management System (CEMS)

This chapter explains the principles that guide disciplined software development and the characteristics that make software different from many physical products.

1. Learning Objectives

By the end of this chapter, you should be able to:

- Explain why software engineering needs principles rather than coding alone.
- Describe core principles such as abstraction, modularity, separation of concerns, simplicity, reuse, and traceability.
- Explain common characteristics of software and distinguish software from hardware.
- Relate quality attributes such as reliability, usability, maintainability, security, and efficiency to real examples.
- Apply these ideas to the College Event Management System.
- Review the main concepts from Module I covered in Days 1–8.

> **Key idea**
> Software engineering principles guide how we make design and development decisions. Software characteristics and quality attributes help us understand what software is like and how well it must perform.

2. Why Do We Need Software Engineering Principles?

A small program may be written by one person in a short time. A real application, however, may be used by thousands of people, contain many modules, store sensitive data, integrate with external systems, and need support for years. Without disciplined practices, changes become risky, defects are difficult to locate, and different parts of the system may behave inconsistently.

Principles are general guidelines for making sound decisions. They do not replace requirements, architecture, testing, or professional judgment. Instead, they help a team decide how to organize complexity, manage change, and build a system that can be understood and maintained.

3. Core Software Engineering Principles

3.1 Abstraction

Abstraction means focusing on the essential features of something while hiding unnecessary detail. A student using an event registration screen sees the event name, available seats, and Register button; the student does not need to know how the database transaction or notification service works.

Why it helps: abstraction reduces the amount of detail a person must understand at one time and creates clearer interfaces between parts of a system.

3.2 Modularity

Modularity means dividing a system into smaller components or modules with defined responsibilities. CEMS might have separate Event Management, Registration, Notification, and Attendance modules.

Why it helps: modules can be developed, tested, understood, and changed more independently—provided their interfaces and dependencies are designed carefully.

3.3 Separation of Concerns

Separation of concerns means keeping different responsibilities distinct. For example, user-interface code should not contain all database access rules, security checks, and business calculations in one large block.

Why it helps: a change to how a screen looks should not unnecessarily require rewriting the rules that decide whether a student is eligible to register.

3.4 Simplicity (KISS)

Simplicity means choosing the clearest design that satisfies the real requirements. Extra layers, abstractions, and features increase the amount of code and behavior the team must understand and maintain.

Example: if a college only needs a simple event list and registration, building a complex recommendation engine before it is needed adds avoidable cost and risk.

3.5 Reuse and Avoiding Duplication

Reuse means using an existing component, library, or well-tested solution when it fits the requirement. Avoiding duplication means not copying the same business rule into many places that can drift apart.

Caution: reuse is valuable only when the reused solution is suitable, secure, supported, and understood. Reusing an unsuitable component can create more problems than it solves.

3.6 Traceability

Traceability means being able to connect a requirement to design decisions, implementation, and tests. If the requirement says “students must not register twice for the same event,” the team should be able to identify the related design rule, code, and test cases.

Why it helps: traceability supports impact analysis, validation, audits, and confidence that important requirements have been addressed.

3.7 Design for Change and Maintainability

Software requirements often change. A maintainable design localizes likely changes, uses clear names, avoids unnecessary coupling, and includes appropriate tests and documentation. The goal is not to predict every future change, but to avoid making ordinary changes unnecessarily expensive.

3.8 Verify Early and Often

Reviews and tests should happen throughout development, not only at the end. Reviewing requirements, testing small units, integrating frequently, and checking user workflows can reveal defects before they spread.

4. Summary Table: Principles in Practice

Principle	CEMS example	Main benefit
Abstraction	A registration screen hides database details.	Reduces unnecessary complexity.
Modularity	Registration and notifications have defined responsibilities.	Supports understanding and controlled change.
Separation of concerns	UI, business rules, and data access are separated.	Avoids tangled responsibilities.
Simplicity	Build only the features justified by requirements.	Reduces needless cost and complexity.
Reuse	Use a suitable, maintained authentication library.	Avoids reinventing well-solved functionality.
Traceability	Link duplicate-registration requirement to tests.	Shows whether requirements are covered.
Maintainability	Keep event-capacity rules in a clear, testable location.	Makes safe changes easier.
Early verification	Test registration limits while the feature is developed.	Finds problems sooner.

5. Software Characteristics

Software has properties that differ from many physical products. Understanding these characteristics helps explain why software projects require disciplined design, testing, configuration management, and maintenance.

5.1 Software Is Developed, Not Manufactured

The major cost of software is usually in analysis, design, implementation, testing, and maintenance. Once a software product is built, making another copy usually costs very little compared with manufacturing another physical device. However, creating the original reliable product can be expensive.

5.2 Software Does Not Physically Wear Out

Unlike a mechanical part, software does not wear out because it has been used many times. However, it can fail because of defects, changing environments, increased load, corrupted data, expired certificates, unsupported dependencies, or changes that introduce new faults. Maintenance and operational care remain essential.

5.3 Software Is Intangible and Complex

Software is not a physical object that can be inspected by touch. Its structure exists in code, data, configuration, and behavior. Even a seemingly small application can have many possible paths, states, and interactions, which makes reasoning and testing challenging.

5.4 Software Can Be Changed, but Change Has a Cost

Software is often modified to meet new needs, correct defects, or support new environments. Although a code edit may be small, its impact can spread to interfaces, data, security, tests, documentation, and other modules. Good design and regression testing help manage this risk.

5.5 Software Is Often Customizable and Reusable

Software can be configured or extended for different users and organizations. Components and libraries may be reused across products, but integration, licensing, compatibility, and security need to be considered.

6. Software Quality Characteristics

Software characteristics describe broad properties of software; quality attributes describe qualities stakeholders expect from a system. The terms can overlap. The following are important quality attributes to consider when engineering software.

Quality attribute	Meaning	CEMS example
Correctness / functionality	The system provides the required behavior.	Registration is blocked when an event is full.
Reliability	The system operates consistently under stated conditions.	Registration remains dependable during peak demand.
Usability	Users can learn and operate the system effectively.	Students can register without confusing steps.
Efficiency / performance	The system uses resources well and responds in acceptable time.	Event pages load quickly for many users.
Security	The system protects data and operations from unauthorized access or misuse.	Only authorized coordinators can export participant lists.
Maintainability	The system can be understood, corrected, and enhanced efficiently.	A new attendance rule can be changed and tested safely.
Portability / compatibility	The system can work across intended environments or interact with other systems.	The student portal works on supported browsers and phones.
Scalability	The system can handle growth in users or workload with an appropriate approach.	The portal can handle festival registration peaks.

A useful distinction

A feature can work correctly in one test and still be unreliable under heavy load, difficult to use, insecure, or hard to maintain. Quality must be considered from multiple perspectives.

7. Putting the Concepts Together

Consider a new requirement: “Students must not register for an event after it reaches capacity.”

1. **Requirement:** record the rule clearly and define what should happen when the event is full.
2. **Abstraction:** expose a simple registration outcome to the user without revealing internal database details.
3. **Modularity and separation of concerns:** keep capacity validation in an appropriate business-rule component, not scattered across unrelated screens.
4. **Traceability:** connect the requirement to design, implementation, and test cases.
5. **Correctness testing:** verify that registration is accepted when seats are available and rejected when the event is full.
6. **Reliability and concurrency:** test simultaneous registrations so the system does not exceed capacity.
7. **Usability:** show a clear message when registration is unavailable.
8. **Maintainability:** make the capacity rule understandable and safe to modify.

This example shows that principles and quality attributes are connected. Principles guide the way the system is designed and developed; quality attributes describe important outcomes the system should achieve.

8. Classroom Activity: Diagnose a Poorly Designed System

Scenario: One large component handles login, event listings, registration, email notifications, and database access. The same capacity-checking rule is copied into three screens. There are no automated tests, and the team cannot identify which tests cover each requirement.

Discuss in pairs:

- Which software engineering principles are being violated or neglected?
- What problems might occur when the capacity rule changes?
- Which quality attributes are at risk?
- Suggest three improvements that would make the system easier to change and test.

9. Module I Review

You have now studied the introductory concepts and the main process models. Use this table to recall the distinctive emphasis of each model.

Topic / model	Main takeaway
Nature of software and software crisis	Software is complex, intangible, changeable, and can fail when projects lack disciplined processes.
SDLC	A lifecycle organizes work such as planning, requirements, design, implementation, testing, deployment, and maintenance.
Waterfall	Sequential phases; often fits stable, well-understood requirements.
Incremental	Product functionality is delivered in planned portions.
Spiral	Repeated cycles explicitly focus on risk analysis.
Prototype	An early representation helps clarify requirements or test an idea.
Agile	Collaboration, frequent feedback, iterative delivery, and adaptation.

10. Knowledge Check

1. What is abstraction? Give a CEMS example.
2. How does modularity help software development?
3. Explain the difference between software being developed and manufactured.
4. Does software physically wear out? What can cause it to fail over time?
5. Name four software quality attributes and explain each briefly.
6. What is traceability, and why is it useful?
7. How are software engineering principles connected to quality attributes?
8. Which process model explicitly emphasizes risk analysis?

Answer key

1. Hiding unnecessary detail; for example, a registration screen hides database operations. 2. It separates responsibilities and can make modules easier to understand and change. 3. Software is created through engineering activities; copies generally cost little to reproduce compared with physical manufacturing. 4. Software does not physically wear out, but defects, environment changes, load, dependencies, and updates can cause failures. 5. Examples: reliability, usability, security, maintainability, performance. 6. Linking requirements to design, implementation, and tests. 7. Principles guide design and development decisions that support desired qualities. 8. Spiral Model.

11. Key Terms

Term	Meaning
Abstraction	Focusing on essential features while hiding unnecessary detail.
Modularity	Dividing a system into components with defined responsibilities.
Separation of concerns	Keeping distinct responsibilities separate.
Traceability	Connecting requirements to design, implementation, and tests.
Maintainability	How effectively software can be understood, corrected, and changed.
Reliability	Ability to operate consistently under stated conditions.
Usability	How effectively users can learn and operate the system.
Scalability	Ability to accommodate growth in workload or users.

12. Homework / Preparation for Module II

For a College Event Management System, prepare a one-page table containing:

- Four functional requirements and four non-functional requirements.
- At least three software engineering principles that would guide the design.
- Four quality attributes and how you would evaluate them.
- One example showing how a requirement could be traced to a test case.

Next module
Day 9 begins Module II — Requirements Engineering, starting with the requirements engineering process and the role of stakeholders.

Syllabus alignment: Module I — Software Engineering Principles and Software Characteristics, with a review of SDLC and Software Process Models. Examples are illustrative teaching scenarios.